Tugas 2 - Membuat Sistem Informasi Manajemen Data Sederhana

Kasus : Pendaftaran Calon Peserta Didik (Sekolah)

Nama: Rizki Mardiansyah

NIM : 10124036

Kelas: IF-1

Username dan Password untuk masuk ke dashboard :

Username : admin

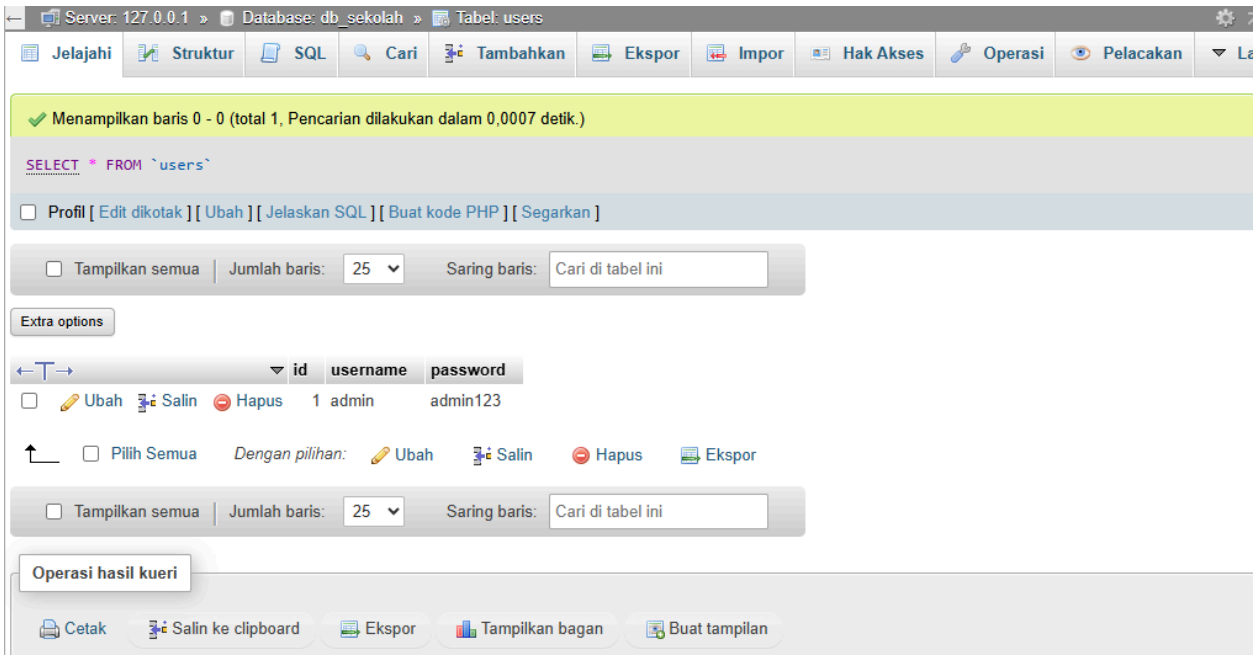
Password : admin123

1. Skema Database (MySQL)

Sebelum menulis kode PHP, langkah pertama yang saya lakukan adalah merancang dan membuat database di MySQL (melalui phpMyAdmin) dengan nama **db_sekolah**. Skrip pembuatannya saya simpan pada file `database/db_sekolah.sql` sehingga database dapat diimpor ulang di komputer lain. Di dalam database ini terdapat dua tabel utama agar sistem berjalan sesuai ketentuan tugas.

A. Tabel users

Tabel ini dipakai untuk sistem otentikasi (login). Kolomnya terdiri dari `id` (Primary Key, Auto Increment), `username` (UNIQUE), dan `password`. Data akun admin dimasukkan lewat perintah INSERT pada skrip SQL, sehingga pengguna dapat langsung login setelah database diimpor.

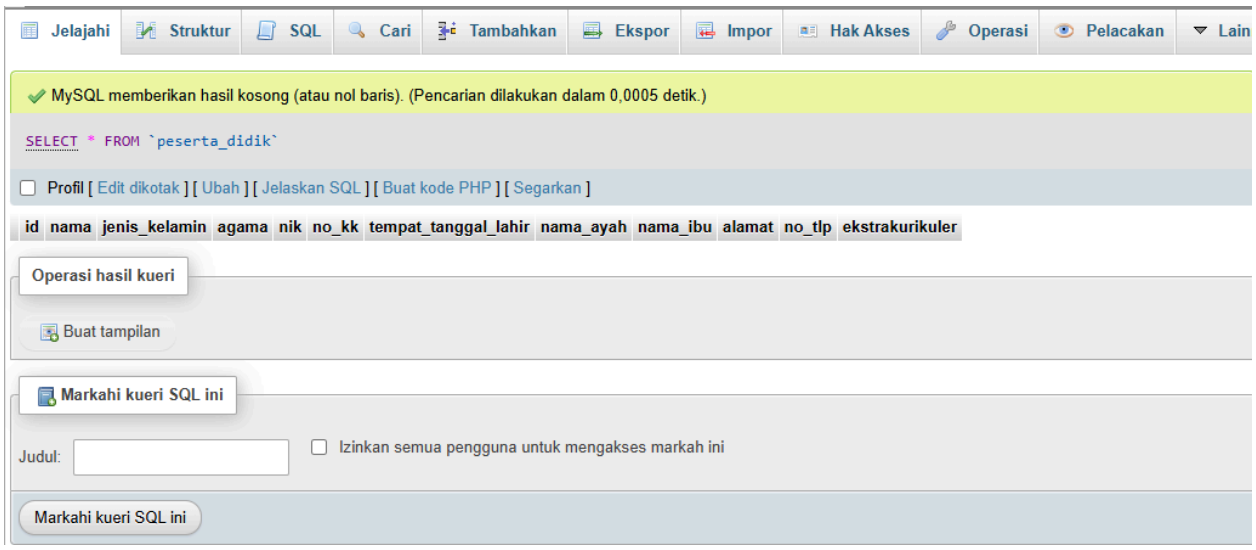


Gambar Database Tabel Users

B. Tabel peserta_didik

Tabel ini dipakai untuk proses CRUD data calon peserta didik. Struktur kolomnya sebagai berikut.

Kolom	Tipe Data	Keterangan
id	INT	Primary Key, Auto Increment
nama	VARCHAR(100)	Nama lengkap, NOT NULL
jenis_kelamin	VARCHAR(20)	Laki-laki / Perempuan
agama	VARCHAR(50)	Islam, Kristen, Katolik, Hindu
nik	VARCHAR(20)	Nomor Induk Kependudukan
no_kk	VARCHAR(20)	Nomor Kartu Keluarga
tempat_tanggal_lahir	VARCHAR(100)	Tempat dan tanggal lahir
nama_ayah	VARCHAR(100)	Nama ayah
nama_ibu	VARCHAR(100)	Nama ibu
alamat	TEXT	Alamat lengkap
no_tlp	VARCHAR(20)	Nomor telepon / HP
ekstrakurikuler	VARCHAR(50)	Pramuka, Paskibra, PMR, Futsal



Gambar Database Tabel Peserta Didik

2. Alur Integrasi & Koneksi Database

Setelah database siap, saya membuat file `koneksi.php`. File ini menjadi jembatan yang menghubungkan skrip PHP dengan database db_sekolah. Saya menggunakan antarmuka **PDO (PHP Data Objects)** yang lebih modern dibanding mysqli biasa, karena PDO mendukung *Prepared Statements* untuk keamanan.

Alur pembuatannya dimulai dengan mendefinisikan variabel DSN (*Data Source Name*) yang memuat informasi host, nama database, dan charset. Kemudian, eksekusi koneksi (`new PDO`) diletakkan di dalam blok `try-catch`. Mode error PDO diatur menjadi `ERRMODE_EXCEPTION` agar kesalahan mudah ditangani. Jika koneksi gagal (misalnya server MySQL mati), error akan ditangkap oleh `catch` dan program dihentikan dengan fungsi `die()`. File `koneksi.php` ini kemudian dipanggil di semua modul lain menggunakan `require 'koneksi.php'`.



```php
<?php
$host     = "localhost";
$nama_db  = "db_sekolah";
$username = "root";
$password = "";
$dsn      = "mysql:host=" . $host . ";dbname=" . $nama_db . ";charset=utf8";

try {
    $pdo = new PDO($dsn, $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Koneksi gagal: " . $e->getMessage());
}
```

Gambar Koneksi PHP

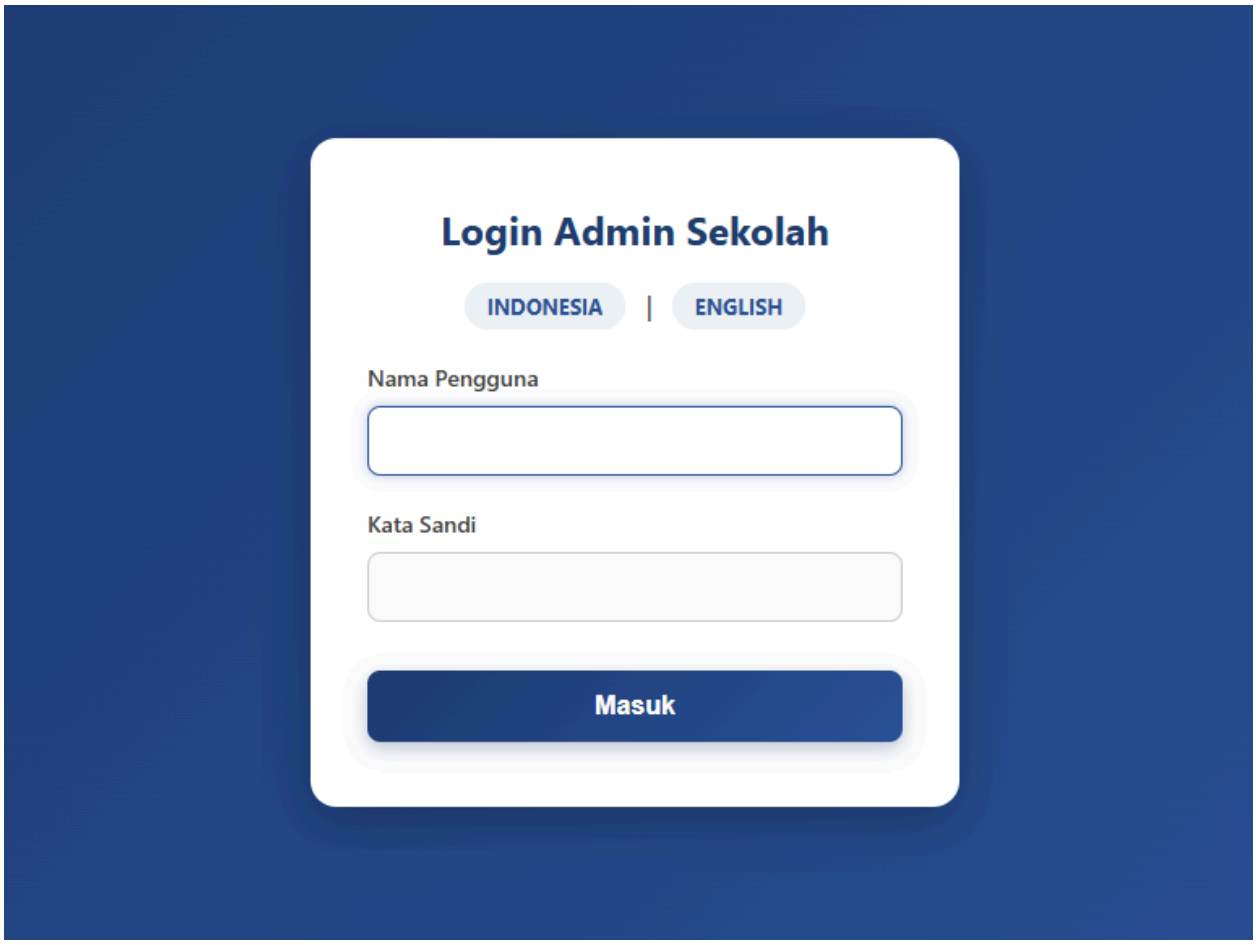
3. Halaman Login

Halaman ini dibuat untuk mengunci modul CRUD agar hanya admin yang punya akses. `session_start()` dipanggil di awal skrip agar program dapat menggunakan variabel superglobal `$_SESSION`. Form login memakai `method="POST"` supaya username dan password yang diketik tidak terlihat di URL.

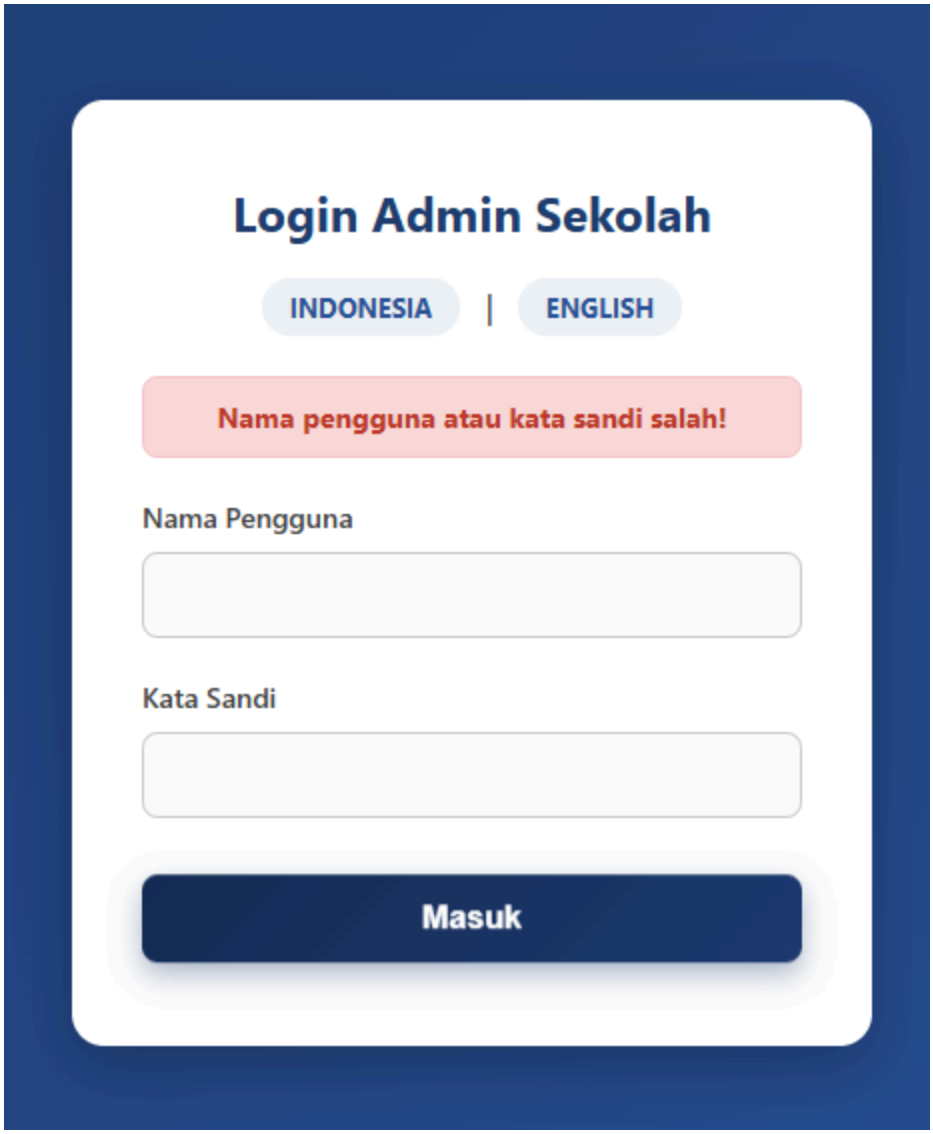
Alur logikanya: saat tombol Masuk diklik, PHP mengecek apakah form dikirim lewat `isset($_POST['login'])`. Jika ya, username dan password dicocokkan dengan tabel `users` menggunakan *Prepared Statement* (`SELECT * FROM users WHERE username = ? AND password = ?`) sehingga aman dari SQL Injection. Struktur percabangan if-else dipakai di sini: jika data ditemukan, program membuat `$_SESSION['status_login'] = true` lalu mengarahkan pengguna ke `index.php`

dengan `header("Location: index.php")`. Jika tidak cocok, variabel `$error` diisi pesan "Nama pengguna atau kata sandi salah!" yang kemudian ditampilkan di atas form.

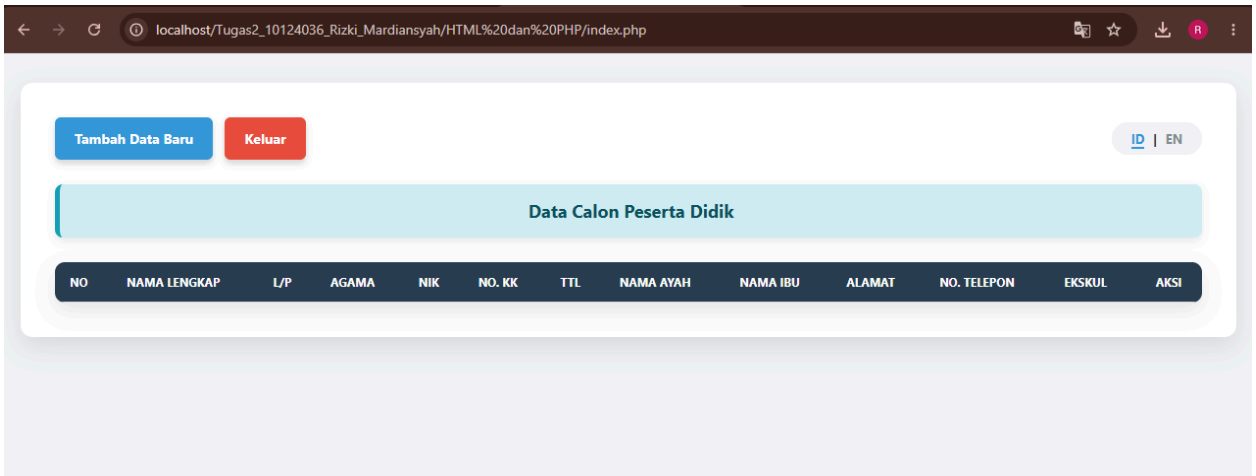
Setiap halaman CRUD (index, tambah, edit, hapus) diawali dengan pengecekan `if (!isset($_SESSION['status_login']))`. Jika sesi login tidak ada, pengguna otomatis dialihkan kembali ke `login.php`, sehingga halaman CRUD tidak dapat diakses tanpa otentikasi.



Gambar Halaman Login



Gambar Halaman Login Saat Gagal



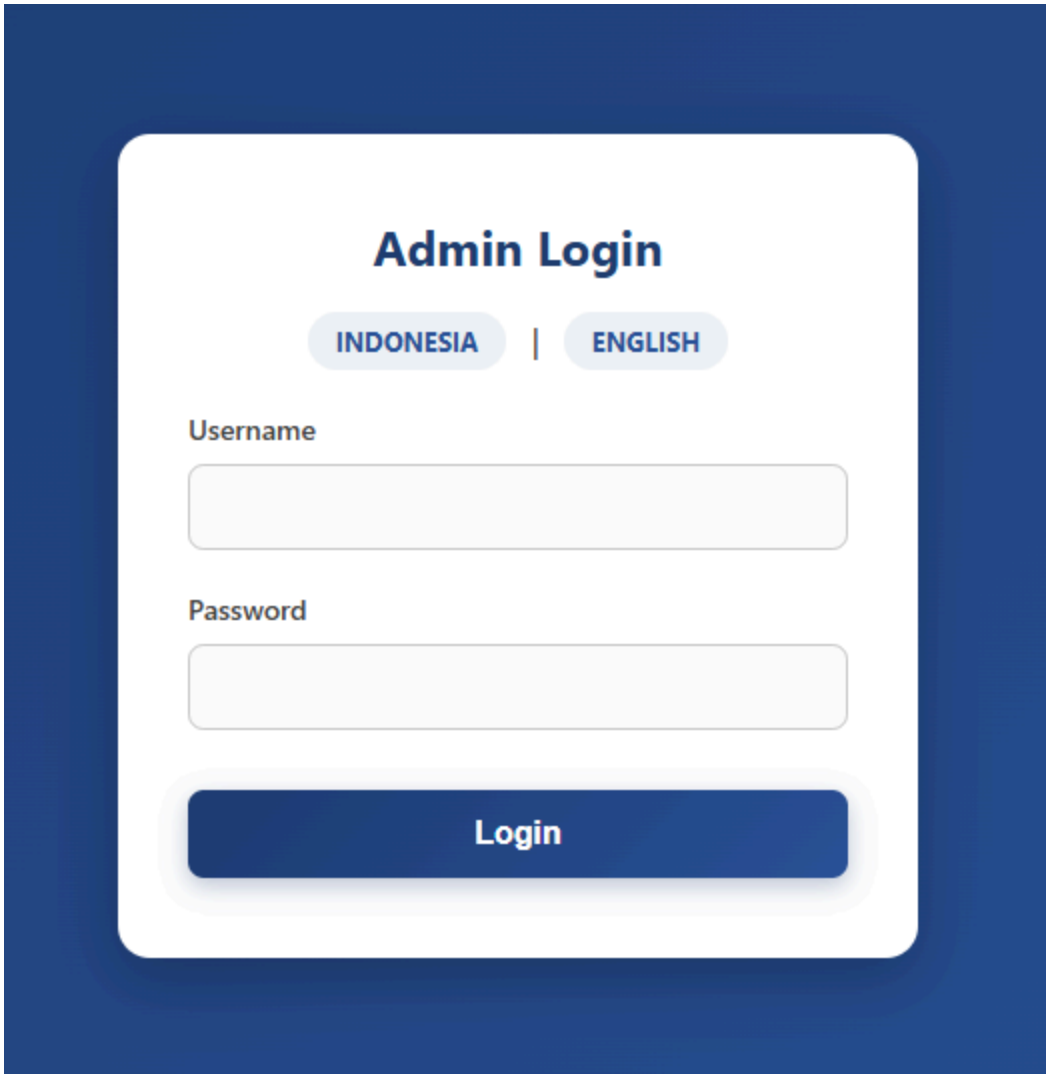
Gambar Proteksi Halaman Tanpa Login

4. Fitur Dual Bahasa (Session)

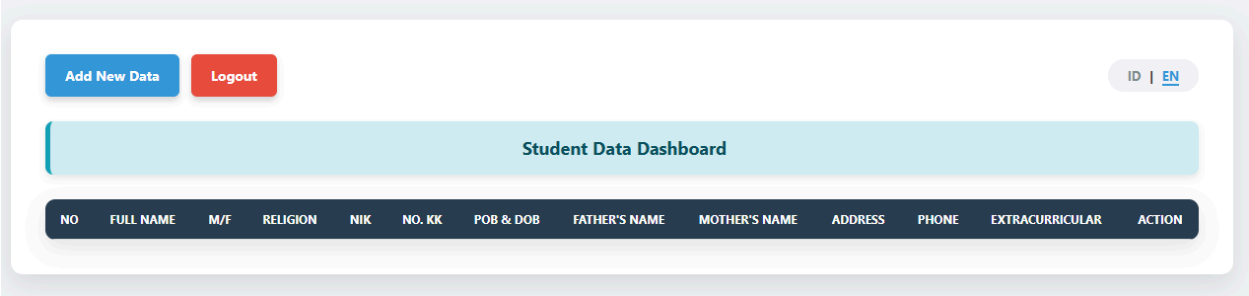
Pilihan bahasa antarmuka (Bahasa Indonesia dan Bahasa Inggris) dikelola oleh file terpisah `kamus_bahasa.php` yang dipanggil di setiap halaman dengan `require 'kamus_bahasa.php'`. Alurnya sebagai berikut.

Pertama, pengguna menekan tautan ID atau EN yang mengirim parameter `?lang=id` atau `?lang=en`. Kedua, skrip menangkap parameter tersebut dan menyimpannya ke `$_SESSION['lang']`. Ketiga, variabel `$lang` diambil dari session dengan nilai bawaan `id` (`$_SESSION['lang'] ?? 'id'`). Keempat, seluruh teks antarmuka (label form, judul tabel, tombol, pesan notifikasi, dan pesan error) disimpan dalam variabel kamus seperti `$t_login`, `$t_user`, dan `$t_btn_save` dengan operator ternary: `($lang == 'en') ? 'Save Data' : 'Simpan Data'`.

Karena pilihan bahasa disimpan di `$_SESSION`, bahasa yang dipilih tetap konsisten ketika pengguna berpindah antar halaman (login, dashboard, tambah, dan edit) tanpa perlu memilih ulang.



Gambar Halaman Login Bahasa Inggris



Gambar Dashboard Bahasa Inggris

5. Halaman Dashboard (Read)

Begitu berhasil login, pengguna diarahkan ke Dashboard (`index.php`). Halaman ini mewakili modul **Read** dari CRUD, yaitu membaca data dari database dan menampilkannya dalam bentuk tabel HTML yang rapi.

Data diambil dengan query `SELECT * FROM peserta_didik ORDER BY id DESC` sehingga data yang paling baru ditambahkan tampil di baris paling atas. Hasil query disimpan ke array dengan `fetchAll(PDO::FETCH_ASSOC)`, lalu di-render ke tabel menggunakan perulangan `foreach`. Setiap nilai ditampilkan dengan `htmlspecialchars()` untuk mencegah serangan XSS. Kolom nomor urut dibuat dari variabel `$no` yang bertambah di setiap baris.

Di bagian atas halaman terdapat tombol **Tambah Data Baru**, tombol **Keluar** (logout), dan pilihan bahasa ID/EN. Tombol Keluar menuju `logout.php` yang menjalankan `session_destroy()` untuk membersihkan sesi lalu mengalihkan pengguna kembali ke halaman login. Banner notifikasi sukses (tambah, ubah, hapus) juga ditampilkan di halaman ini dan otomatis menghilang setelah 10 detik lewat JavaScript.



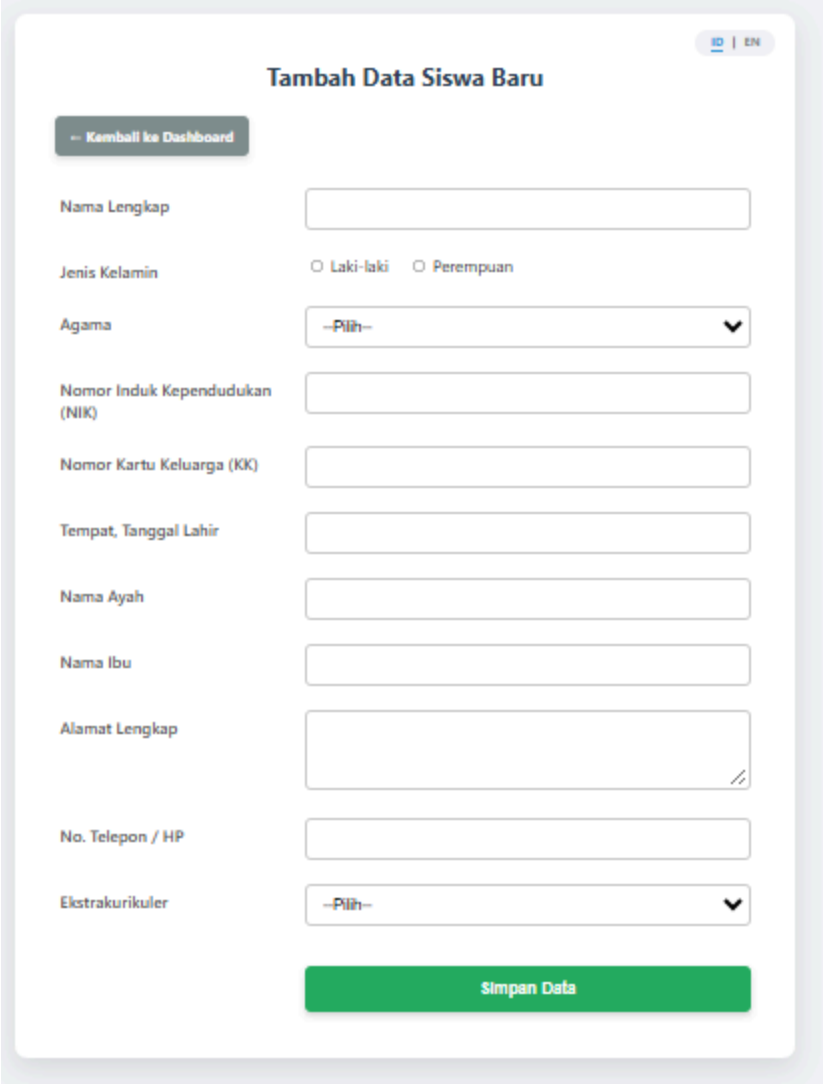
Gambar Halaman Dashboard

6. Halaman Tambah Data (Create & Validasi)

Form pada `tambah.php` mewakili modul **Create**. Form ini mengirim data dengan `method="POST"` ke file itu sendiri. Pilihan input mencakup kolom teks, radio button (jenis kelamin), dropdown (agama dan ekstrakurikuler), serta textarea (alamat).

Alur penyimpanannya: saat tombol Simpan diklik, setiap input langsung disanitasi memakai `htmlspecialchars()` untuk mencegah masuknya skrip berbahaya (XSS). Setelah itu, struktur percabangan `if (empty($nama) || empty($nik) || empty($no_kk) || empty($no_tlp))` memeriksa kelengkapan data wajib. Jika ada yang kosong, variabel `$error_msg` diisi pesan kesalahan dan data tidak disimpan. Form juga memiliki atribut `required` sebagai lapisan validasi di sisi browser.

Jika lolos validasi, data disimpan lewat perintah `INSERT INTO peserta_didik ...` yang dieksekusi dengan *Prepared Statement* PDO berparameter penanda (seperti `:nama`, `:nik`, `:kk`). Ini mencegah serangan SQL Injection. Setelah berhasil, program menyimpan penanda `$_SESSION['pesan_sukses'] = 'added'` lalu mengalihkan pengguna ke dashboard yang menampilkan pesan "Data berhasil ditambahkan!".



Gambar Halaman Tambah Data



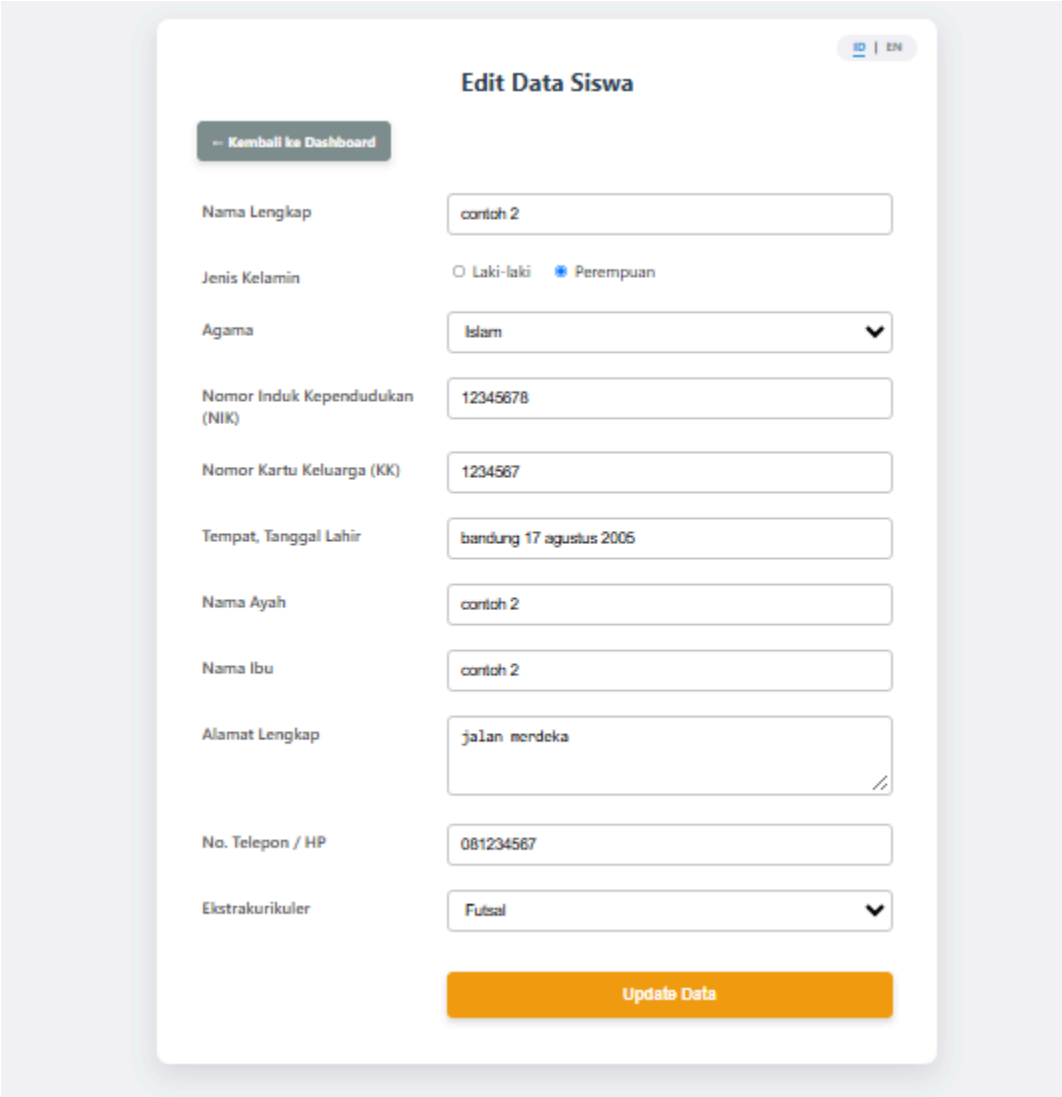
Gambar Halaman Dashboard yang Berhasil Menambahkan Data

7. Halaman Ubah Data (Update)

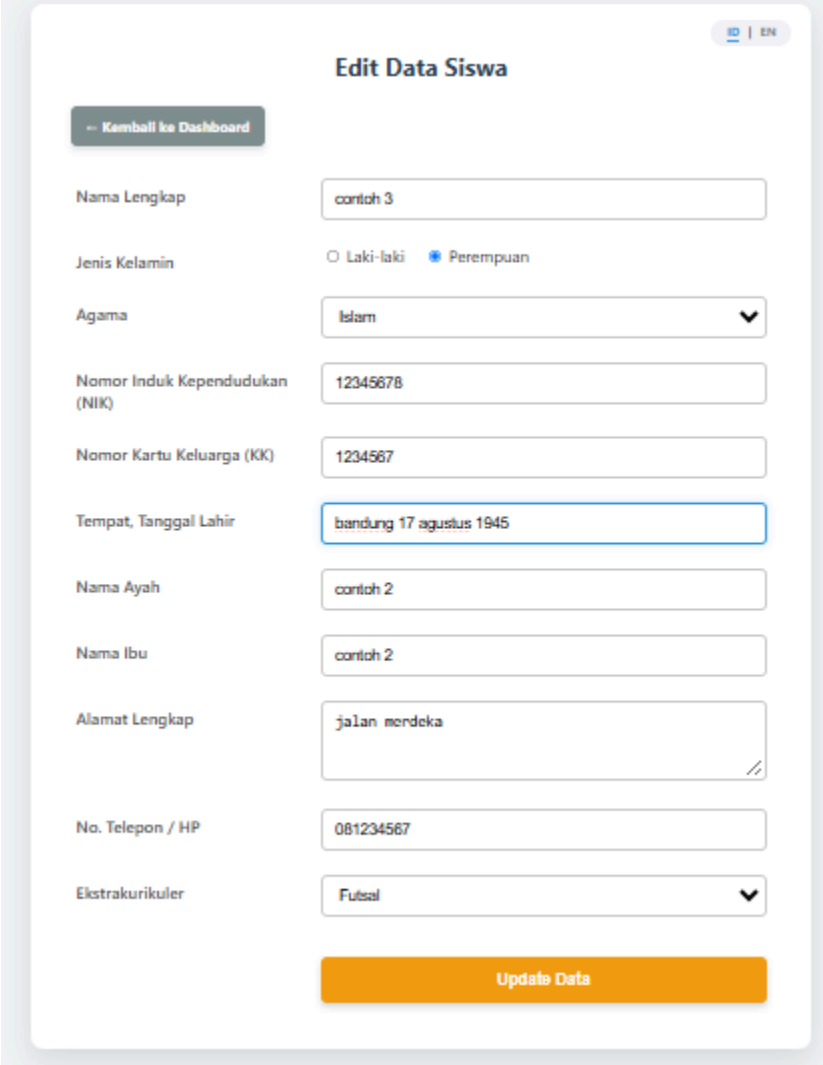
Halaman `edit.php` mewakili modul **Update**. Secara tampilan mirip dengan form Tambah, tetapi alur awalnya berbeda. Saat tombol Edit di baris tabel diklik, URL membawa ID data tersebut (contohnya `edit.php?id=5`). Skrip PHP menangkap parameter itu lewat `$_GET['id']`, lalu menjalankan query `SELECT * FROM peserta_didik WHERE id = :id` untuk menarik data lama dan mengisikannya ke dalam form (*pre-filled*). Jika parameter ID tidak ada, pengguna dikembalikan ke dashboard. Jika ID tidak ditemukan di database, muncul pesan "Data tidak ditemukan!".

Setelah pengguna mengubah nilai dan klik Update, proses sanitasi dan validasi yang sama seperti form Tambah dijalankan. Bedanya, query yang dieksekusi adalah `UPDATE peserta_didik SET ... WHERE id=:id` dengan *Prepared Statement* PDO. Jika sukses, pengguna diarahkan ke dashboard dengan pesan "Data berhasil diperbarui!".

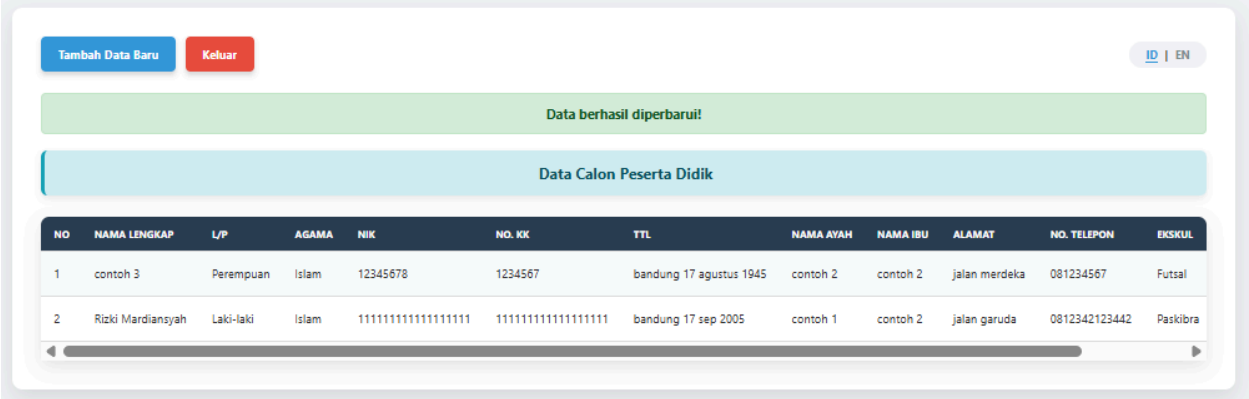
[TEMPEL SCREENSHOT 12: halaman Edit Data sebelum diubah (form terisi data lama)]



Gambar Halaman Data Sebelum di Update



Gambar Halaman Data yang Akan di Update

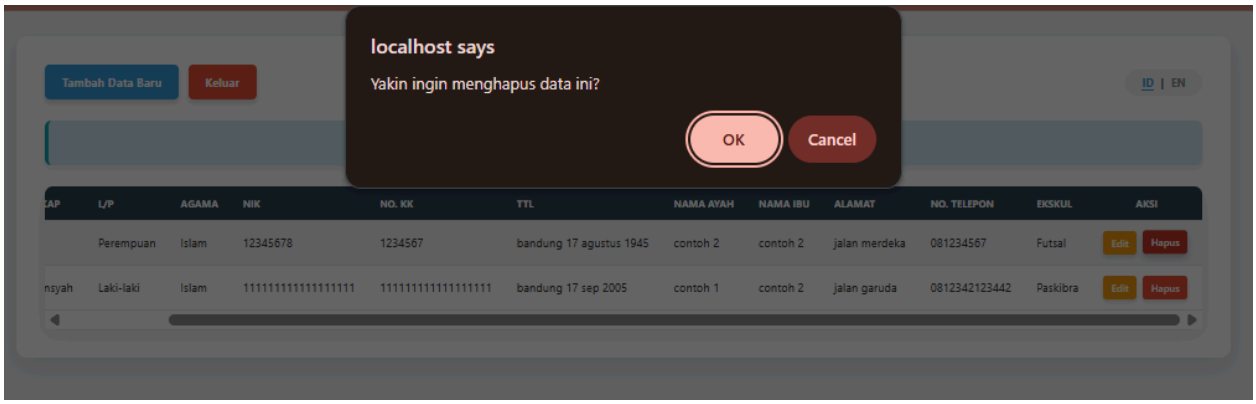


Gambar Halaman Dashboard Setelah Data di Update

8. Fitur Hapus Data (Delete)

Untuk modul **Delete**, tidak dibuat halaman HTML khusus agar prosesnya lebih cepat. Saat pengguna menekan tombol Hapus pada salah satu baris di dashboard, muncul jendela *pop-up* konfirmasi dari JavaScript (`return confirm('Yakin ingin menghapus data ini?')`). Ini mencegah data terhapus karena klik yang tidak disengaja.

Jika pengguna memilih OK, program berpindah ke `hapus.php` dengan membawa ID data (misal `hapus.php?id=3`). Skrip tersebut memvalidasi session, menangkap parameter ID, lalu menjalankan `DELETE FROM peserta_didik WHERE id = :id` menggunakan PDO *Prepared Statement*. Setelah data terhapus, penanda `$_SESSION['pesan_sukses'] = 'deleted'` disimpan dan `header("Location: index.php")` mengembalikan pengguna ke dashboard yang menampilkan pesan "Data berhasil dihapus!".



Gambar Halaman Dashboard yang Akan Datanya Dihapus



Gambar Halaman Dashboard yang Datanya Berhasil Dihapus

9. Struktur File Proyek

File	Fungsi
database/db_sekolah.sql	Skrip ekspor database (tabel users dan peserta_didik)
koneksi.php	Koneksi database menggunakan PDO
kamus_bahasa.php	Kamus terjemahan ID/EN dan penyimpanan bahasa di session
login.php	Halaman login dan pembuatan session
logout.php	Menghapus session dan kembali ke login
index.php	Dashboard, menampilkan data (Read)
tambah.php	Form tambah data (Create) dan validasi
edit.php	Form ubah data (Update)
hapus.php	Proses hapus data (Delete)